KAALMO

Complete System Workflow Documentation

Somalia-First Fundraising Platform

System Architecture · User Flows · Payment Flow · Security Flow · Data Flow · Admin Flow

Prepared from: **kaalmo-web-only-spec.md** (authoritative source)
Document type: Software System Architecture & Workflow Reference

Table of Contents

1. Introduction 4
2. System Overview 4
Master System Tree 5
3. System Architecture 6
Technology Stack (MVP) 7
4. User Roles 7
5. Master System Workflow 8
6. Authentication Workflow 10
7. Verification Workflow 11
8. Organizer Workflow 11
9. Co-organizer Workflow 13
10. Beneficiary Workflow 14
11. Donor Workflow 14
12. Campaign Lifecycle 16
13. Donation Workflow 17
14. Payment Architecture 19
15. Financial Ledger Workflow 20
16. Withdrawal Workflow 21
17. Trust & Safety Workflow 23
18. Admin Workflow 24
19. Moderator Workflow 24
20. Support Workflow 25
21. Database / Data Relationships 25
Collection Purposes 26
22. API Request Flow 27
Representative Endpoints 28
23. Security Architecture 29
24. Notification Workflow 30

25. Deployment Architecture 31

MVP Deployment (Current) 31

Future Production-Scale Architecture (Not Current) 31

26. End-to-End Example 32

27. Error & Alternative Flows 33

28. Complete Master Diagram 34

29. Glossary 36

30. Final System Summary 37

1. Introduction

Kaalmo is a Somalia-first digital fundraising platform, conceptually similar to GoFundMe but built around how Somalis actually raise and give money: through trusted community networks, mobile money, diaspora remittance habits, and word-of-mouth verification rather than institutional trust. Today, most Somali fundraising happens informally — WhatsApp forwards, Facebook posts, hawala transfers to a relative's number — which offers no transparency, no proof funds were used correctly, and no way for a diaspora donor abroad to verify a campaign before sending money.

This document explains, from beginning to end, how the Kaalmo system is designed to work: how a user enters the platform, how a campaign is created and reviewed, how a donation moves from a donor's phone to a confirmed financial record, and how that money eventually reaches a verified beneficiary through the withdrawal process. It is written for developers, system architects, project managers, QA engineers, security engineers, investors, and future team members who need a single, authoritative reference for how the product is intended to behave.

This document is derived strictly from **kaalmo-web-only-spec.md**, the platform's product and technical specification. Where the specification leaves a detail open — a specific payment provider, an exact withdrawal threshold, a specific fraud-scoring algorithm — this document says so explicitly rather than inventing an answer.

2. System Overview

Kaalmo connects three kinds of people around a campaign: an **Organizer** who creates and runs a fundraiser, a **Beneficiary** who the money is actually for, and **Donors** who give money to support the campaign. Around this core relationship sits a trust layer (verification, moderation, fraud review) and a financial layer (payments, ledger, withdrawals) that together make sure the platform can say, with evidence, that a campaign is real and that its money is accounted for.

Master System Tree

```
KAALMO
|
+-- USERS
|   +-- Donor
|   +-- Organizer
|   +-- Beneficiary
|   +-- Co-organizer
|   +-- Admin
|   +-- Moderator / Trust & Safety
|   +-- Support Agent
|
+-- AUTHENTICATION & VERIFICATION
|   +-- Registration
|   +-- Login (JWT access + refresh)
|   +-- Email Verification
|   +-- Phone Verification (OTP)
|   +-- Identity Verification
|   +-- Beneficiary Verification
|   +-- Payment Verification
|   +-- Organization Verification
|
+-- CAMPAIGN SYSTEM
|   +-- Create Campaign (Draft)
|   +-- Submitted
|   +-- Under Review
|   +-- Approved
|   +-- Published
|   +-- Active
|   +-- Goal Reached
|   +-- Withdrawal
|   +-- Completed
|   +-- (side states: Rejected, Suspended, Frozen,
|        Cancelled, Expired)
|
+-- DONATION SYSTEM
|   +-- Discover Campaign
|   +-- Campaign Details
|   +-- Donation Request
|   +-- Payment Intent
|   +-- Payment Provider
|   +-- Webhook
|   +-- Payment Confirmation
|   +-- Donation Confirmation
|   +-- Ledger (PaymentTransaction)
|   +-- Receipt
|
+-- TRUST & SAFETY
|   +-- Verification
|   +-- Reports
|   +-- Fraud Detection (FraudCase)
|   +-- Risk Scoring (LOW/MEDIUM/HIGH/CRITICAL)
|   +-- Moderation
|   +-- Freeze
|   +-- Suspension
|
+-- WITHDRAWAL
|   +-- Request
```

```
|   +-- Beneficiary Verification Check
|   +-- Admin / Risk Review
|   +-- Approval
|   +-- Payment Provider (Payout)
|   +-- Payout Confirmation
|   +-- Completed
|
+-- ADMIN
    +-- Users
    +-- Campaigns / Campaign Review
    +-- Donations
    +-- Payments
    +-- Withdrawals
    +-- Verification
    +-- Fraud / Risk
    +-- Reports
    +-- Audit Logs
    +-- System Settings
```

Figure 1 — Master System Tree

The complete set of subsystems that make up Kaalmo, grouped the way the specification groups them: Users, Authentication & Verification, the Campaign system, the Donation system, Trust & Safety, Withdrawal, and Admin.

3. System Architecture

Kaalmo's MVP architecture (Section 6 and Section 32 of the specification) is a standard MERN-style web stack: a React web frontend talking to a Node.js/Express REST API, backed by MongoDB, with a handful of external services for payments, email, SMS, and file storage.

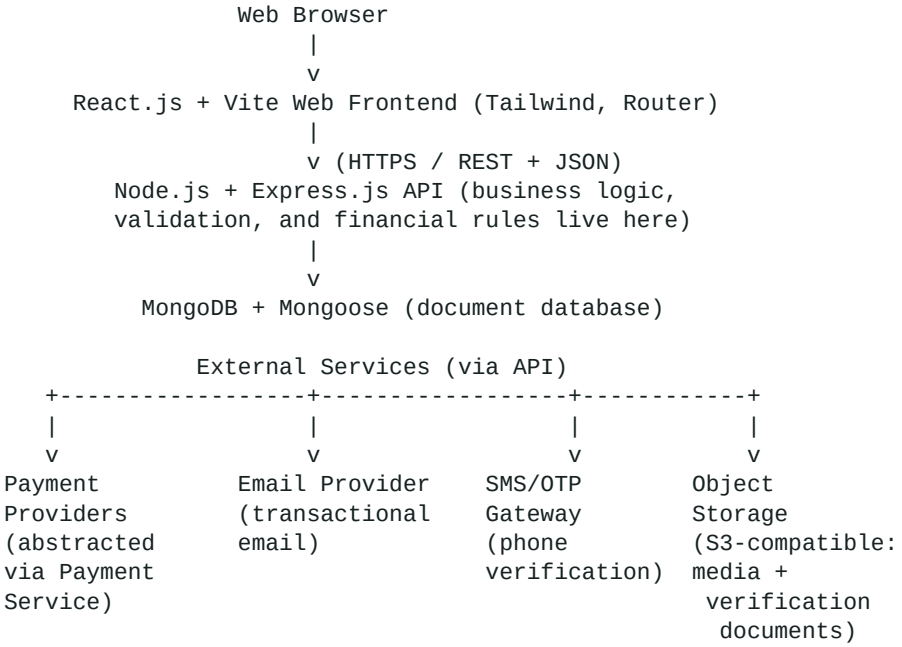


Figure 2 — MVP System Architecture

All business logic, validation, and financial rules live server-side in the Node.js/Express API — the web client is a presentation layer only. An OpenAPI contract keeps the client and backend in sync (specification Section 5).

Technology Stack (MVP)

Layer	Choice	Why
Frontend Web	React.js + Vite, Tailwind CSS, React Router	Fast dev velocity, consistent UI
Backend	Node.js + Express.js	Matches MERN requirement, JSON API development
Database	MongoDB + Mongoose	Flexible schema fits evolving campaign/verification data
Auth	JWT (access + refresh tokens)	Stateless, scales horizontally
File storage	Object storage (S3-compatible)	Campaign media and verification documents
Email	Transactional email provider	Verification, receipts, notifications
SMS/OTP	Local-capable SMS gateway	Phone verification — coverage to be validated

The specification explicitly lists what is **not** in the MVP: Redis, background job queues, push notifications, advanced analytics, multi-region deployment, and GraphQL. These are Future/Scale additions (specification Section 6), added once real usage justifies the operational overhead.

4. User Roles

Every person who uses Kaalmo holds one or more of seven roles. A single account can hold multiple roles at once — for example, the same person is usually both a Donor and, once verified, an Organizer.

Actor	Main Responsibility	Main Access
Donor	Gives money	Public campaigns + own donation history
Organizer	Runs campaigns	Own campaigns, invites co-organizers, requests withdrawal
Co-organizer	Assists organizer	Accepted campaign membership only (not global)
Beneficiary	Receives funds	Own verification + payout account
Admin	Operates platform	Full administration — web dashboard only
Moderator / Trust & Safety	Reviews risk	Campaign/risk review — not financial settings
Support Agent	Customer support	Limited support data — no withdrawal/fraud actions

Each role's key permissions and restrictions, exactly as defined in the specification (Section 4):

Role	Key Permissions	Key Restrictions
Donor	Browse, donate, comment, follow, bookmark, view own history/receipts	Cannot edit campaigns; cannot see other donors' private info
Organizer	Create/edit/publish/pause/close campaign, invite co-organizers, post updates, request withdrawal	Cannot self-verify; cannot withdraw without beneficiary+payment verification
Beneficiary	Accept invitation, submit verification docs, connect payout account, view progress	Cannot edit campaign content unless also organizer
Co-organizer	Post updates, view donations/analytics, edit campaign content	Cannot request withdrawals or delete campaign (organizer-only)
Admin	Full access: users, campaigns, payments, withdrawals, verification, settings	Web dashboard only, no native mobile app
Moderator	Review/approve/reject/freeze campaigns, handle reports, manage fraud queue	Cannot access platform-wide financial settings or fee configuration
Support Agent	View tickets, view limited user/campaign info, respond to inquiries	No access to withdrawal approval or fraud actions

Each role gets a dashboard scoped to only what it needs — organizer dashboards emphasize campaign performance; admin dashboards emphasize risk and financial oversight (specification Section 4).

5. Master System Workflow

This is the complete lifecycle of Kaalmo, from a user's first visit through to money reaching a verified beneficiary. Every later section in this document zooms into one stage of this master flow.

```
USER
 |
 v
REGISTER
 |
 v
VERIFY ACCOUNT  (email + phone)
 |
 v
CREATE CAMPAIGN (organizer)  /  DISCOVER CAMPAIGN (donor)
 |
 v
CAMPAIGN (draft)
 |
 v
SUBMIT
 |
 v
MODERATION / REVIEW
 |
 v
APPROVED
 |
 v
PUBLISHED
 |
```

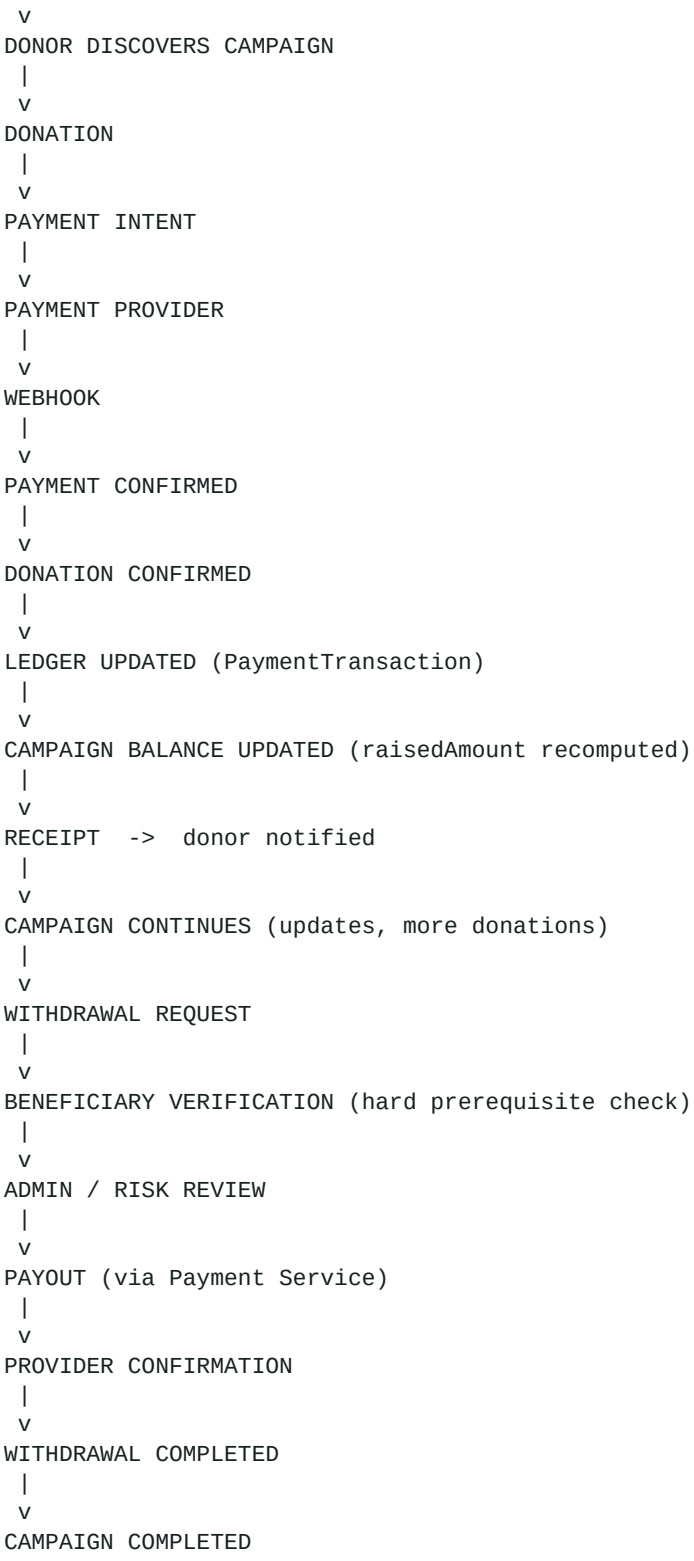


Figure 3 — Master End-to-End Workflow

Derived from the specification's MVP User Flow (Section 29): Register → Create Campaign → Submit → Admin Approves → Publish → Share → Donor Opens Campaign → Donate → Payment Gateway → Webhook → Donation Confirmed → Campaign Balance Updated → Organizer Dashboard → Withdrawal Request → Admin Review → Payout.

Why this order matters: a campaign cannot receive donations until it is Published; a donation cannot be counted until a payment provider (not the browser) confirms it; and a withdrawal cannot be requested until the beneficiary is verified. Each of these gates exists to protect the same thing — donor trust that money given is money that actually reaches a real cause.

6. Authentication Workflow

Authentication establishes *who* is making a request. Kaalmo uses JWT-based authentication with a short-lived access token (~15 minutes) and a longer-lived refresh token, rotated on use (specification Section 19).

```
REGISTER  (email, phone, password)
 |
 v
ACCOUNT CREATED  (passwordHash via bcrypt/argon2)
 |
 v
EMAIL VERIFICATION  (signed, time-limited link)
 |
 v
PHONE VERIFICATION  (OTP, rate-limited, expiring)
 |
 v
LOGIN
 |
 v
ACCESS TOKEN (JWT, ~15 min)
 |
 v
REFRESH TOKEN (long-lived, rotated on use,
                httpOnly secure cookie for web)
 |
 v
AUTHENTICATED REQUEST
 |
 v
AUTHORIZATION  (role + ownership check)
```

Figure 4 — Authentication Workflow

Concept	How it works per the specification
Password hashing	bcrypt/argon2 — never reversible (Section 19)
Access token	Short-lived (~15 min), JWT (Section 19)
Refresh token	Long-lived, rotated on use, stored httpOnly secure cookie for web (Section 19)
Email verification	Signed, time-limited link (Section 19)
Phone verification	OTP, rate-limited and expiring (Section 19)
Password reset	Signed, single-use token, short expiry; invalidates active sessions on completion (Section 19)
Session management	List active sessions, allow remote logout, refresh-token invalidation on password change (Section 19)

■ *The specification does not define the exact OTP length, the specific email/SMS provider, or the precise brute-force lockout thresholds — only that rate limiting, progressive lockout, and CAPTCHA-after-repeated-failures are required (Section 20).*

7. Verification Workflow

Verification is Kaalmo's core trust mechanism. It is a tiered chain — each tier builds on the one before it, and higher tiers unlock higher donation/withdrawal limits and "Featured" eligibility (specification Section 22).

```
Email Verified
      |
      v
Phone Verified
      |
      v
Identity Verified
      |
      v
Beneficiary Verified   <-- hard prerequisite for Withdrawal
      |
      v
Payment Verified
      |
      v
Organization Verified

  Higher tier -> higher donation/withdrawal limits
                 + "Featured" eligibility
```

Figure 5 — Verification Tier Chain

Verification badges are displayed on campaign cards and detail pages, e.g. "✓ Identity Verified ✓ Beneficiary Verified ✓ Payment Verified" (specification Section 22).

Verification is both a trust signal for donors and a risk-control lever for the platform: it directly gates what an account is allowed to do next — most importantly, **Beneficiary Verified** is a hard prerequisite before any withdrawal can be requested (specification Section 15).

8. Organizer Workflow

The organizer is the person who creates and runs a fundraiser. The specification's organizer journey (Section 10) is: Register → Verify account → Create fundraiser → Add info → Upload media → Add beneficiary → Submit → Verification → Approval → Publish → Share → Receive donations → Post updates → Request withdrawal → Receive funds.

```
Register
 |
 v
Verify account (email + phone)
 |
 v
Create campaign
 |
 v
```

```
Add:
 +-- Title
 +-- Story
 +-- Goal amount
 +-- Category
 +-- Media (images/video)
 +-- Beneficiary
 +-- Other required information
 |
 v
Save Draft
 |
 v
Submit
 |
 v
Under Review  (moderator checks documents/story/beneficiary)
 |
 v
Approved  -----------------+
 |                         |
 v                         v (if failed review)
Published              Rejected -> organizer fixes -> resubmit
 |
 v
Share Campaign (WhatsApp / social)
 |
 v
Receive Donations
 |
 v
Post Updates
 |
 v
Monitor Analytics
 |
 v
Request Withdrawal
 |
 v
Review (beneficiary verification + admin/risk review)
 |
 v
Payout
 |
 v
Completed
```

Figure 6 — Organizer Workflow

Example — Hodan in Hargeisa organizes fundraisers for her extended family and community (specification Section 2, persona). She registers, verifies her account, creates a campaign for a relative's medical treatment, adds the relative as the beneficiary, and submits it for review. Once a moderator approves it, she publishes it and shares the link on WhatsApp — the platform's dominant traffic channel (specification Section 37).

Friction points the specification calls out (Section 10): verification can feel slow or bureaucratic — mitigated with clear progress indicators and expected review time; uploading documents on slow connections

— mitigated with client-side compression and resumable upload; uncertainty about "did my campaign go live?" — mitigated with proactive SMS/push at every status change.

9. Co-organizer Workflow

The co-organizer model is **campaign-scoped**, not a global role. An organizer invites a specific user to help with one specific campaign; that user becomes a co-organizer of *that campaign only*, tracked through the CampaignMembers collection (specification Sections 4 and 16).

```
User (any registered account)
      |
      v
Organizer invites User to a specific Campaign
      |
      v
Invitation created  (CampaignMembers: invitedAt)
      |
      v
User Accepts  (CampaignMembers: acceptedAt)
      |
      v
CampaignMembers record: { campaignId, userId,
                          role: 'co-organizer' }
      |
      v
CO-ORGANIZER  (scoped to this one campaign only)
      |
      v
Allowed campaign actions:
   +-- Post updates
   +-- View donations
   +-- View analytics
   +-- Edit campaign content
      |
      v
NOT allowed (organizer-only):
   +-- Request withdrawal
   +-- Delete campaign

Relationship:
  User -> Campaign -> CampaignMembers -> Co-organizer

Security boundary: a co-organizer is NOT a global
organizer. Access is granted per campaign, not per
account -- accepting one invitation does not grant
access to any other organizer's campaigns.
```

Figure 7 — Co-organizer Invitation & Access Model

CampaignMembers schema (specification Section 16): { campaignId, userId, role: 'co-organizer', invitedAt, acceptedAt }

Security boundary: per the specification's permission table (Section 4), a co-organizer can **post updates, view donations/analytics, and edit campaign content** for the campaign they were invited to. They explicitly **cannot request withdrawals or delete the campaign** — those actions remain organizer-only. The specification's authorization model (Section 19) requires fine-grained permission checks — "can this user edit *this specific* campaign" — not just a coarse role check, which is exactly what the campaign-scoped

CampaignMembers relationship is designed to enforce.

The testing strategy (specification Section 31) explicitly calls for a negative test case: *"co-organizer cannot withdraw"* — underscoring that this boundary is a first-class security requirement, not an incidental detail.

10. Beneficiary Workflow

The beneficiary is the person or institution the funds are actually for — sometimes the same person as the organizer, sometimes not (e.g. a family member organizes a campaign on behalf of a sick relative). The specification's beneficiary feature list (Section 7) is: accept invitation; verify identity; connect payout account; view progress; view withdrawal status.

```
Organizer creates campaign
 |
 v
Beneficiary added  (Campaign.beneficiaryId)
 |
 v
Beneficiary receives invitation
 |
 v
Accept invitation
 |
 v
Submit verification documents  (idDocumentUrl)
 |
 v
Identity verification
 |
 v
Beneficiary verification  (verificationStatus, verifiedBy)
 |
 v
Connect payout account  (PayoutAccount: mobile money or bank)
 |
 v
Wait for withdrawal eligibility
 |
 v
Withdrawal  (organizer/beneficiary can now request payout)
```

Figure 8 — Beneficiary Workflow

Why beneficiary verification matters before withdrawal: the specification states plainly that "Beneficiary verification is a hard prerequisite — no withdrawal can be requested until beneficiary identity + payout account are verified" (Section 15). This exists because the beneficiary — not necessarily the organizer — is who the money is meant to reach; without verifying that this is a real person/institution with a real, matching payout account, the platform cannot responsibly release donor funds.

11. Donor Workflow

The donor journey (specification Section 10) is: Discover → View campaign → Read story → Check verification → Donate → Payment → Confirmation → Receipt → Follow → Receive updates.

```
Visit platform
 |
 v
Register / Login
 |
 v
Browse campaigns
 |
 v
Search / Filter  (category, region, tags)
 |
 v
Open campaign
 |
 v
Review:
 +-- Story
 +-- Organizer identity
 +-- Verification badges
 +-- Progress (raised / goal)
 +-- Updates
 |
 v
Donate  (amount, optional message, optional anonymous)
 |
 v
Payment  (provider-specific: mobile money / card / bank)
 |
 v
Confirmation  (webhook-driven, not frontend-driven)
 |
 v
Receipt
 |
 v
Donation History  (donor's own record)
 |
 v
Follow / Bookmark / Receive Updates
```

Figure 9 — Donor Workflow

What a donor can do (specification Section 4): browse, donate, comment, follow, bookmark, and view their own donation history/receipts. Donations may be made anonymously, with an optional message (Section 7). **What a donor cannot do:** edit campaigns, or see other donors' private information.

Example — Yusuf in Minneapolis (specification Section 2, persona) sends money home regularly via hawala and wants to donate by card, see proof a campaign is real, and get updates on impact. He discovers a medical campaign, checks its verification badges and organizer identity, donates $50 by card, receives an instant confirmation and receipt, and follows the campaign to see future updates on how the funds are used.

Friction points the specification calls out (Section 10): trust hesitation before donating — mitigated with prominent badges, visible organizer identity, and real update history; payment failure anxiety with mobile money — mitigated with a clear retry flow and instant SMS confirmation; wanting to donate anonymously without losing receipt access — supported as "anonymous-to-public but authenticated-to-platform."

12. Campaign Lifecycle

Every campaign moves through a defined state machine (specification Section 9). The happy path is linear; several side states exist for when things go wrong or need to pause.

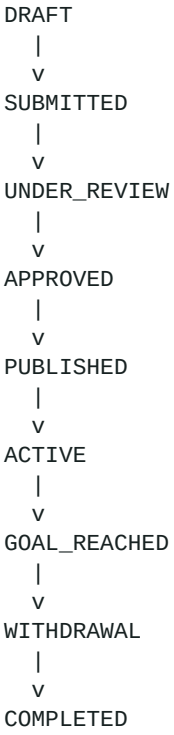


Figure 10 — Campaign Lifecycle — Main Path

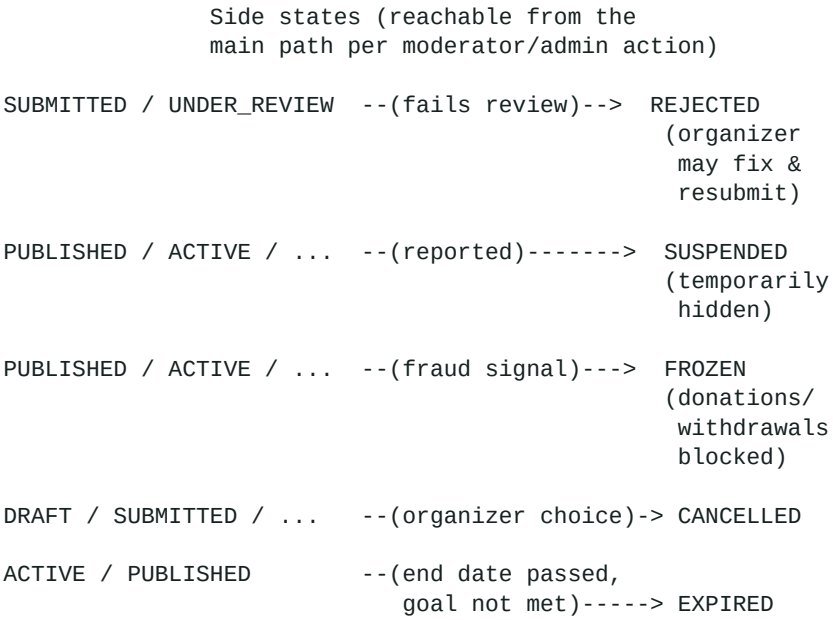


Figure 11 — Campaign Lifecycle — Side States

REJECTED, SUSPENDED, FROZEN, CANCELLED, and EXPIRED are reachable from the main path but are not part of the happy-path progression (specification Section 9).

Status	Meaning (per specification)
DRAFT	Organizer is still editing; not visible to anyone else
SUBMITTED	Submitted for review; locked from further edits except minor fields
UNDER_REVIEW	Moderator actively checking documents/story/beneficiary
APPROVED	Passed review, ready to go live
PUBLISHED	Live and publicly visible, may still be pre-donation
ACTIVE	Actively receiving donations
GOAL_REACHED	Funding goal met (can still receive donations unless organizer closes it)
WITHDRAWAL	Beneficiary/organizer has requested payout, admin reviewing
COMPLETED	Funds disbursed, campaign archived as successful
REJECTED	Failed review with reason shown to organizer, can resubmit after fixes
SUSPENDED	Temporarily hidden due to a report/investigation
FROZEN	Donations/withdrawals blocked pending fraud investigation
CANCELLED	Organizer withdrew the campaign voluntarily
EXPIRED	Passed end date without reaching goal or being closed

■ *The specification does not enumerate, status-by-status, exactly which role can trigger every individual transition, nor the exact reachability rules between every pair of states (e.g. whether SUSPENDED can transition back to ACTIVE). Section 24 confirms that admin/moderator actions include review/approve/reject/suspend/freeze/restore, with mandatory reason codes on every reject/suspend action, but the full transition matrix is left to implementation.*

13. Donation Workflow

A donation is the core transaction of the platform. The specification is explicit that payment confirmation must never be determined by the frontend — it is always the payment provider's webhook, verified server-side, that marks money as real.

```
Donor
 |
 v
Campaign  (must be PUBLISHED / ACTIVE)
 |
 v
Donation Request  (amount, message, anonymous flag)
 |
 v
Backend Validation  (schema validation, campaign eligible)
 |
 v
Payment Intent  (created via PaymentService)
 |
 v
Payment Service  (routes to correct provider adapter)
```

```
 |
 v
Payment Provider  (STK-push-style prompt for mobile money)
 |
 v
Payment Confirmation  (provider-side, out of band)
 |
 v
Webhook  (provider calls Kaalmo backend)
 |
 v
Signature Verification  (reject if invalid)
 |
 v
Idempotency Check  (dedupe by provider transaction ID)
 |
 v
PaymentTransaction  (created, immutable ledger entry)
 |
 v
Donation Confirmed
 |
 v
Campaign Raised Amount  (recomputed from ledger)
 |
 v
Receipt  (generated for donor)
 |
 v
Notification  (donor + organizer notified)
```

Figure 12 — Detailed Donation Workflow

"Payment Intent created → Donor redirected/prompted to provider (STK-push-style for mobile money) → Provider webhook confirms → PaymentTransaction marked confirmed → Donation marked confirmed → Campaign raisedAmount incremented → Receipt generated → Donor notified." (specification Section 14)

Why the webhook matters: mobile money confirmation can take 10–60 seconds (specification Section 14). A donor's browser cannot know, on its own, whether the donor actually entered their PIN and the provider actually moved money — only the provider does. The webhook is the provider telling Kaalmo's backend, out-of-band and with a verifiable signature, that the payment genuinely succeeded.

Why the frontend must never determine payment success: a browser response can be spoofed, intercepted, or simply lost if the user closes the tab mid-payment. If the frontend alone decided "payment succeeded," any donor could fabricate a successful donation without paying. The specification's security architecture (Section 20) requires "webhook signature verification on every payment provider callback + replay protection via idempotency keys" specifically to close this gap.

Donation — Alternative Flows

```
Donation Payment States
     |
     +-- PENDING   (mobile money confirmation, 10-60s)
     |
     +-- CONFIRMED (webhook verified -> ledger updated)
     |
     +-- FAILED    (clear retry UX shown to donor)

Webhook Duplicate Delivery
     |
     +-- Same provider transaction ID seen again
     |         |
     |         v
     |    Idempotency check matches existing record
     |         |
     |         v
     |    No duplicate PaymentTransaction created
```

Figure 13 — Donation Payment States

The specification requires the system to handle pending payments, failed payments (with clear retry UX), and idempotency for webhooks that may fire more than once (Section 14).

14. Payment Architecture

Payments are the highest-risk, highest-priority part of the system (specification Section 14). Kaalmo does not talk to any payment provider directly from its core donation logic — every provider sits behind a single abstraction, PaymentService.

```
                  PaymentService (interface)
                           |
    +------------+----------+----------+------------+
    |            |                     |            |
    v            v                     v            v
MobileMoney   BankProvider        CardProvider   ManualProvider
Provider      (local bank         (international (admin-recorded
(EVC Plus /   transfer/           card processor  offline
eDahab /      collection)         with Somalia    donation --
Zaad --                           reach --        MVP fallback)
verify                            verify)
current API
availability)

Common interface implemented by every provider:
  createPaymentIntent()  confirmPayment()
  handleWebhook()        refund()
```

Figure 14 — Payment Abstraction Layer

Each provider implements a common interface: createPaymentIntent(), confirmPayment(), handleWebhook(), refund(). The core donation flow only ever calls PaymentService — adding a new provider requires zero changes to campaign/donation logic (specification Section 14).

```
Donation
   |
   v
PaymentService          (never bypassed by core donation logic)
```

```
   |
   v
Provider Adapter        (e.g. MobileMoneyProvider)
   |
   v
External Provider       (actual mobile money / card / bank rail)
   |
   v
Provider Response
   |
   v
Webhook                 (provider -> Kaalmo backend)
   |
   v
Backend                 (signature check + idempotency check)
   |
   v
PaymentTransaction      (immutable ledger entry)
   |
   v
Donation                (marked confirmed)
```

Figure 15 — Payment Data Flow

■ *The specification explicitly states: "No specific payment provider integration details are assumed here — mobile money aggregators, bank APIs, and card processors available in Somalia should be verified directly with current provider documentation before implementation, since availability, fees, and KYC requirements change." (Section 14). EVC Plus, eDahab, and Zaad are named only as examples of mobile money providers to evaluate, not as confirmed integrations.*

The specification requires the system to handle: pending payments (10–60s mobile-money confirmation delay), failed payments (clear retry UX), idempotency (dedupe webhooks by provider transaction ID), signature verification on every webhook, refunds (organizer/admin-initiated, pre-withdrawal only), chargebacks (card only), multi-currency (USD primary), and fee transparency (show the donor exactly what platform fee applies before confirming) — specification Section 14.

15. Financial Ledger Workflow

Kaalmo's financial integrity rests on one non-negotiable rule, stated directly in the specification (Section 14): **"PaymentTransaction records are append-only/immutable — corrections are made via new reversal/adjustment records referencing the original, never by editing or deleting a transaction row."** This is required for audit and dispute resolution.

```
CONFIRMED TRANSACTION  (provider webhook verified)
        |
        v
Ledger  (PaymentTransaction -- append-only, immutable;
         corrections = new reversal/adjustment records,
         never edits or deletes)
        |
        v
Donation  (status set to confirmed)
        |
        v
Campaign Balance  (raisedAmount)
```

```
  raisedAmount is a DERIVED, recomputable field.
  The SOURCE OF TRUTH is always the sum of confirmed
  PaymentTransactions -- never the stored number itself.
```

Figure 16 — Financial Ledger Chain

The specification is explicit about what is the source of truth versus what is derived (Section 17): "the campaign's raisedAmount is a derived, recomputable field, never the source of truth — the source of truth is the sum of confirmed PaymentTransactions." In other words, a campaign's displayed "raised" total is always a calculation, not a stored fact — it can and should be recomputed from the immutable ledger of confirmed transactions to catch and correct any drift.

The *Definition of Done — MVP Beta-Ready* (Section 40) reinforces this: "Campaign balance updates correctly and reconciles with the payment ledger (no drift between raisedAmount and confirmed PaymentTransactions)."

16. Withdrawal Workflow

Once a campaign has raised funds, the organizer or beneficiary can request a withdrawal — but only after the beneficiary is verified. The specification's withdrawal flow (Section 15) is:

```
Campaign
 |
 v
Funds Available  (raisedAmount from confirmed ledger)
 |
 v
Beneficiary Verified?
 |
 v YES
Withdrawal Request  (by Organizer or Beneficiary)
 |
 v
Risk / Admin Review
 |         (auto-approve under threshold,
 |          manual review above threshold)
 v
Approved
 |
 v
PaymentService
 |
 v
Payout Provider  (mobile money or bank payout)
 |
 v
Provider Confirmation  (webhook)
 |
 v
Withdrawal COMPLETED
 |
 v
Ledger Updated
 |
 v
Campaign Financial State Updated
```

Figure 17 — Withdrawal Workflow — Happy Path

"Campaign raises funds → Beneficiary verification confirmed → Organizer/Beneficiary submits withdrawal request → Admin/risk review (auto-approve under threshold + manual above threshold) → Payout routed via PaymentService to chosen payout method → Payout confirmed by provider webhook → Withdrawal marked COMPLETED, campaign ledger updated." (specification Section 15)

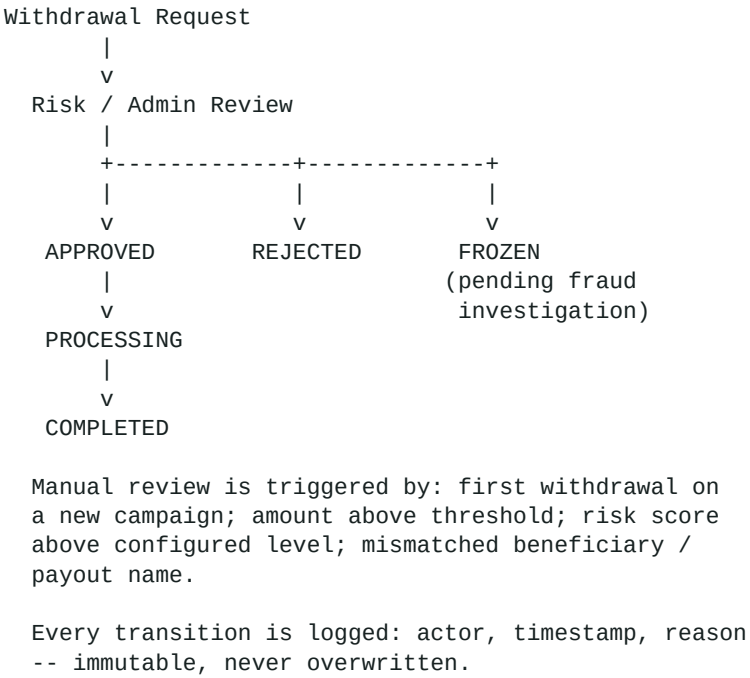


Figure 18 — Withdrawal — Review Outcomes

Concept	Per the specification (Section 15)
Payout methods	Mobile money payout (registered number); bank payout (larger/institutional withdrawals)
Hard prerequisite	Beneficiary identity + payout account must be verified before any request
Withdrawal limits	Configurable per-transaction and daily/weekly caps; tighter for new accounts, loosened after track record
States	pending, under_review, approved, processing, completed, failed, frozen
Manual review triggers	First withdrawal on a new campaign; amount above threshold; risk score above configured level; mismatched beneficiary/payout name
Audit trail	Every state transition logged with actor, timestamp, and reason — immutable, never overwritten

■ *The specification does not state the exact dollar threshold for auto-approval versus mandatory manual review, nor the exact per-transaction/daily/weekly withdrawal-limit figures — these are described as "configurable" (Section 15), meaning they are an operational/business decision, not a fixed rule in the spec.*

17. Trust & Safety Workflow

Trust & Safety is not a one-time check — the specification requires fraud scoring to run "at campaign submission, at each donation above a threshold, and at every withdrawal request — not just once at onboarding" (Section 21).

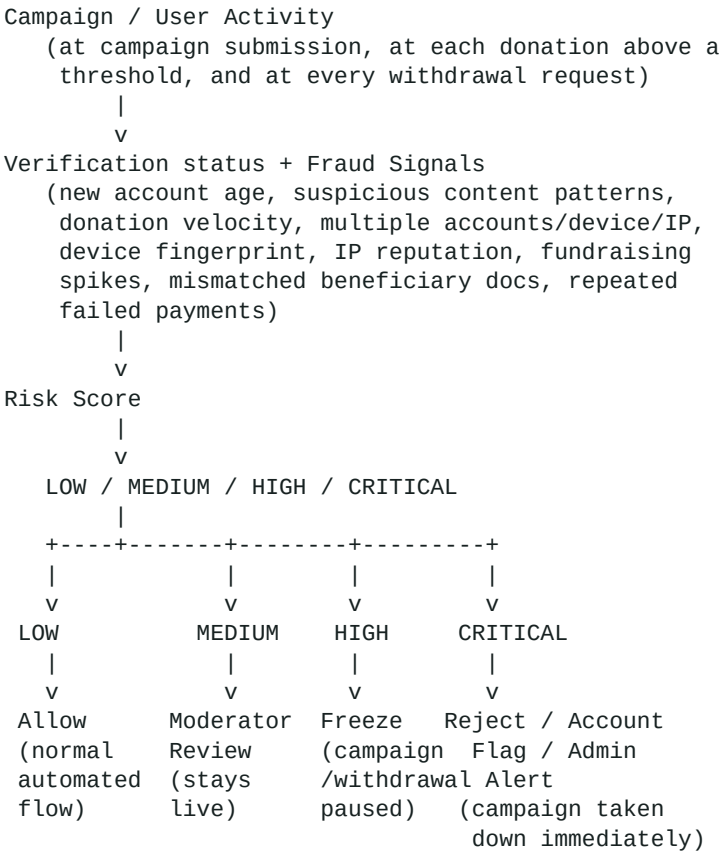


Figure 19 — Fraud & Risk Scoring Workflow

Risk Score	System Action (per specification)
LOW	Allow — normal automated flow
MEDIUM	Review — flagged for moderator queue, campaign stays live
HIGH	Freeze — campaign/withdrawal paused pending manual review
CRITICAL	Reject — campaign taken down, account flagged, admin notified immediately

Fraud signals monitored (specification Section 21): new account age, suspicious campaign content patterns, unusual donation velocity, multiple accounts from same device/IP, device fingerprinting, IP reputation, sudden fundraising spikes inconsistent with campaign history, mismatched beneficiary identity documents, and repeated failed payment attempts.

■ *The specification names the signals monitored and the four-tier score-to-action mapping, but does not define the exact scoring algorithm, weighting, or numeric thresholds that turn signals into a LOW/MEDIUM/HIGH/CRITICAL score.*

18. Admin Workflow

Admin is the platform operator role with full access. The admin dashboard is web-only (specification Sections 4 and 24) — there is no native mobile admin app.

```
ADMIN DASHBOARD
    |
    +-- Users            (search, profile, suspend, ban,
    |                     manual verify)
    +-- Campaigns / Campaign Review
    |                    (approve/reject/suspend/freeze/
    |                     restore, mandatory reason codes)
    +-- Donations        (donations ledger)
    +-- Payments         (payments ledger, failed-txn view)
    +-- Withdrawals      (queue, approve/reject)
    +-- Verification     (identity/beneficiary review queue)
    +-- Fraud / Risk     (FraudCase queue)
    +-- Reports          (user/campaign reports)
    +-- Moderation
    +-- Support          (support tickets)
    +-- Audit Logs       (immutable action history)
    +-- System Settings  (fee configuration, categories)
    +-- Analytics        (KPIs, GMV, success/fraud rates)
```

Figure 20 — Admin Dashboard Areas

specification Section 11 (Pages & Screens Inventory): Dashboard, Users, Campaigns, Campaign Review, Donations, Payments, Withdrawals, Reports, Fraud/Risk, Verification, Categories, Moderation, Support, Audit Logs, System Settings, Analytics.

KPIs shown on the admin dashboard (Section 24): total users, active campaigns, total donations, total volume, pending withdrawals, open fraud cases, open reports, verification queue size.

Moderation: review/approve/reject/suspend/freeze/restore campaigns, with mandatory reason codes on every reject/suspend action. **User management:** search, view profile + activity history, suspend, ban, manually verify, review linked campaigns/donations. **Financial management:** payments ledger, donations ledger, withdrawals queue, fee configuration, refunds, failed transaction investigation view (specification Section 24).

19. Moderator Workflow

Moderator (Trust & Safety) is a narrower role than Admin. Per the role table (Section 4), a moderator can review/approve/reject/freeze campaigns and handle reports/fraud queue, but explicitly **cannot access platform-wide financial settings or fee configuration** — that remains Admin-only.

```
Campaign Submitted
      |
      v
Risk / Trust Review  (documents, story, beneficiary info)
      |
      v
Moderator decision:
   +-- Approve
   +-- Reject     (reason code required)
   +-- Suspend    (reason code required)
```

```
   +-- Freeze     (pending fraud investigation)
   +-- Restore

  Moderator CANNOT access platform-wide financial
  settings or fee configuration -- that remains
  Admin-only.
```

Figure 21 — Moderator Review Workflow

This separation matters for the same reason the co-organizer boundary matters: a role that reviews risk and trust signals should not, by that fact alone, also control the platform's money settings — keeping those permissions apart limits the damage a single compromised or malicious account can do.

20. Support Workflow

The Support Agent role exists for customer support only. Per the specification's role table (Section 4), a support agent can view tickets and limited user/campaign info and respond to inquiries, but has **no access to withdrawal approval or fraud actions**.

```
User submits inquiry
      |
      v
Support Ticket created
      |
      v
Support Agent views ticket
      |         (limited user/campaign info only)
      v
Support Agent responds
      |
      v
Ticket resolved

  Support Agent CANNOT approve withdrawals or take
  fraud actions -- those remain Admin/Moderator-only.
```

Figure 22 — Support Agent Workflow

■ *The specification lists Support Tickets as a standard supporting collection (Section 16) and names the role and its restrictions (Section 4), but does not define a detailed ticket-lifecycle workflow (e.g. specific ticket statuses or escalation rules) beyond "view tickets ... respond to inquiries."*

21. Database / Data Relationships

Kaalmo uses MongoDB with Mongoose. The specification (Section 16) gives a representative schema for each core entity and a relationship tree (Section 17) showing how they connect.

```
User
 |
 +-- Campaigns (organizerId)
 |    +-- Donations (campaignId)
 |    |    +-- PaymentTransaction (paymentTransactionId)
 |    +-- Updates (campaignId)
 |    +-- Comments (campaignId)
 |    +-- CampaignMembers (co-organizers)
 |    +-- Withdrawals (campaignId)
 |
```

```
 +-- Donations (donorId)
 +-- Notifications (userId)
 +-- Reports (reporterId)
 +-- Verifications (userId)

Beneficiary
 |
 +-- Campaign (beneficiaryId)
 +-- PayoutAccount (payoutAccountId)
```

Figure 23 — Database Relationship Tree

specification Section 17. Every financial reference (Donation → PaymentTransaction, Withdrawal → PaymentTransaction) is one-directional and immutable once confirmed.

Collection Purposes

Collection	Purpose (per specification Section 16)
Users	Account, roles, verification flags, status
Campaigns	Core fundraiser record — title/story (so/en), goal, raisedAmount, status, badges
CampaignMembers	Co-organizer relationship: campaignId, userId, role, invitedAt, acceptedAt
Beneficiaries	Person/entity funds are for — verification status, payout account link
Donations	A single gift — amount, currency, anonymity, message, linked payment transaction
Payments / PaymentTransactions / PaymentWebhooks	High-level intent / immutable ledger entries / raw webhook payload + signature result
Withdrawals	Payout request — amount, status, reviewer, provider transaction id
PayoutAccounts	Mobile money or bank account — masked account number, verified flag
Updates / Comments / Likes / Followers / Bookmarks	Campaign engagement, each keyed by campaignId + userId
Notifications	userId, type, payload, channel, read
Reports	reporterId, targetType, targetId, reason, status
FraudCases	subjectType, subjectId, riskScore, signals, status, assignedTo
Verifications	userId/beneficiaryId, verification type, status, documentUrls, reviewedBy
AuditLogs	actorId, actorType, action, targetType, targetId, metadata — immutable, append-only
SupportTickets, Referrals, Fees, Categories	Standard supporting collections

Embed vs. Reference rule (Section 16): embed only when data is small, bounded, and always read with its parent (e.g. a campaign's verificationBadges array). Reference when data grows unboundedly (donations,

comments), is shared across parents (a User referenced by many Donations), or has independent lifecycle/audit needs — **all financial records are always referenced, never embedded**, to preserve immutability and indexability.

22. API Request Flow

Every write to the Kaalmo API passes through the same layered pipeline before it touches business logic or the database (specification Section 18).

```
Frontend  (React web app)
 |
 v
REST API  (/api/v1, Node.js + Express)
 |
 v
Authentication Middleware  (verify JWT)
 |
 v
Authorization Middleware  (role + ownership check)
 |
 v
Validation  (schema-based, e.g. Zod/Joi)
 |
 v
Controller
 |
 v
Service  (business logic -- PaymentService, FraudService, ...)
 |
 v
Database / External Provider
 |
 v
Response  (standardized success or error shape)
```

Figure 24 — API Request Pipeline

"Every write endpoint requires: authentication (JWT), authorization (role/ownership check), input validation (schema-based, e.g. Zod/Joi)" and returns a standardized error shape on failure (specification Section 18).

Standardized error response shape (Section 18): `{ "error": { "code": "VALIDATION_ERROR", "message": "...", "fields": {} } }`

Representative Endpoints

Endpoint	Method	Auth	Purpose
/auth/register	POST	Public	Create account
/auth/login	POST	Public	Get access+refresh tokens
/auth/refresh	POST	Refresh token	Rotate access token
/auth/verify-otp	POST	Public	Confirm phone OTP
/users/me	GET/PATCH	User	View/update own profile
/campaigns	GET	Public	List/search/filter campaigns
/campaigns	POST	Organizer	Create campaign (draft)
/campaigns/:id	GET	Public	Campaign detail
/campaigns/:id	PATCH	Organizer/Co-organizer	Edit campaign
/campaigns/:id/submit	POST	Organizer	Submit for review
/campaigns/:id/publish	POST	Admin	Publish approved campaign
/campaigns/:id/donate	POST	Donor or guest	Create donation + payment intent
/campaigns/:id/updates	GET/POST	Public read / Organizer write	Campaign updates
/campaigns/:id/comments	GET/POST	Public read / Donor write	Comments
/payments/webhook/:provider	POST	Provider signature	Payment confirmation
/withdrawals	POST	Organizer/Beneficiary	Request payout
/withdrawals/:id/review	PATCH	Admin	Approve/reject
/reports	POST	Authenticated user	Report campaign/user
/admin/campaigns/:id/review	PATCH	Admin/Moderator	Approve/reject/suspend
/admin/fraud-cases	GET	Admin	Fraud queue
/notifications	GET	User	List own notifications

(Base path: `/api/v1` — specification Section 18, "representative," not exhaustive.)

23. Security Architecture

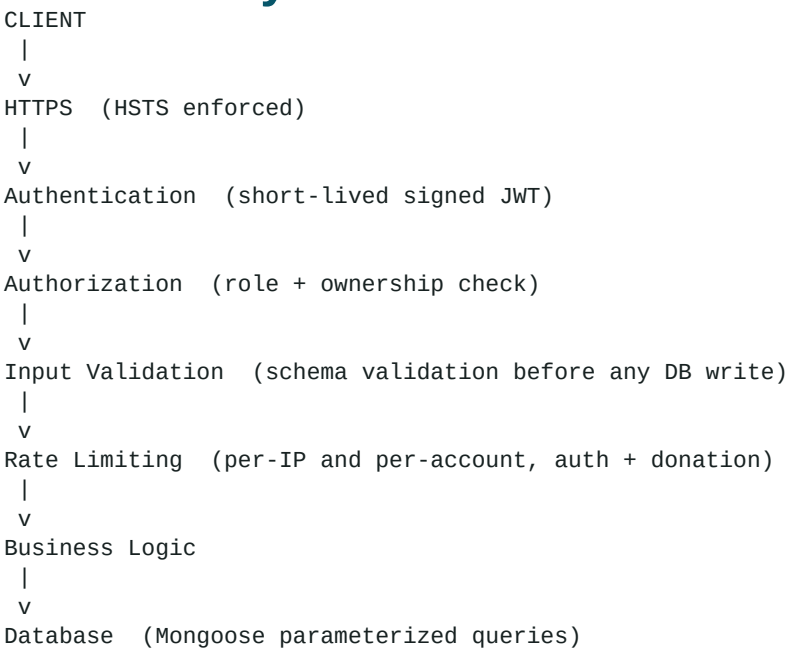


Figure 25 — Security Layers — General Request



Figure 26 — Security Layers — Payment/Webhook Path

The specification's full security control list (Section 20): HTTPS everywhere (HSTS enforced); bcrypt/argon2 password hashing; short-lived signed JWTs with rotation; per-IP and per-account rate limiting on auth and donation endpoints; strict input validation on every endpoint (schema validation before any DB write); Mongoose parameterized queries (no raw query injection); output encoding + CSP headers against XSS; CSRF tokens on cookie-based web sessions; locked-down CORS allowlist; secure, httpOnly, sameSite cookies for refresh tokens; file upload validation (type/size allowlist, virus scan where feasible, stored outside web root, served via signed URLs); webhook signature verification on every payment provider callback + replay protection via idempotency keys; encryption at rest for sensitive fields (ID documents, payout account numbers — masked in UI); secrets in environment/secret manager, never in source control; structured audit

logging of all admin and financial actions; brute-force protection (progressive lockout + CAPTCHA after repeated failures); account takeover protection (notify on new device login, allow session revocation).

Why each layer matters, in the payment path specifically: input validation and rate limiting stop obviously malformed or automated abuse before it reaches business logic; authentication and authorization make sure only the right party can act on a given campaign or donation; and webhook signature verification plus idempotency ensure that only the payment provider itself — never a spoofed or duplicated request — can mark money as confirmed.

24. Notification Workflow

```
SYSTEM EVENT
 (campaign approved/rejected, new donation, goal reached,
  new update, withdrawal approved/completed, payment
  failed, account security alert, new comment)
      |
      v
Notification Service
      |
      v
Channel
   +-- In-App     (MVP: all events)
   +-- Email      (MVP: all events)
   +-- SMS        (MVP: highest-urgency only --
   |                payment confirmation, withdrawal
   |                completed, security alerts)
   +-- Push       (future -- not MVP)
```

Figure 27 — Notification Workflow

MVP channels: In-app + Email for all events; SMS reserved for the highest-urgency events only, to control SMS cost (specification Section 23).

Events (specification Section 23): campaign approved/rejected, new donation, goal reached, new update, withdrawal approved/completed, payment failed, account security alert, new comment.

■ *Push notifications are listed as a channel, but explicitly marked "(future)" in the specification (Section 23) — not part of the current MVP channel set.*

25. Deployment Architecture

MVP Deployment (Current)

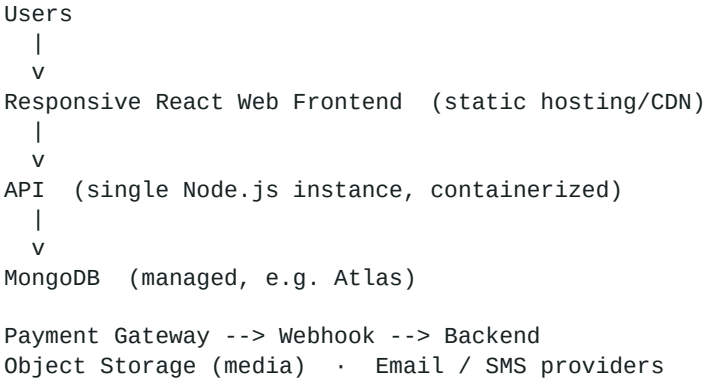


Figure 28 — MVP Deployment Architecture

specification Section 32.

Future Production-Scale Architecture (Not Current)

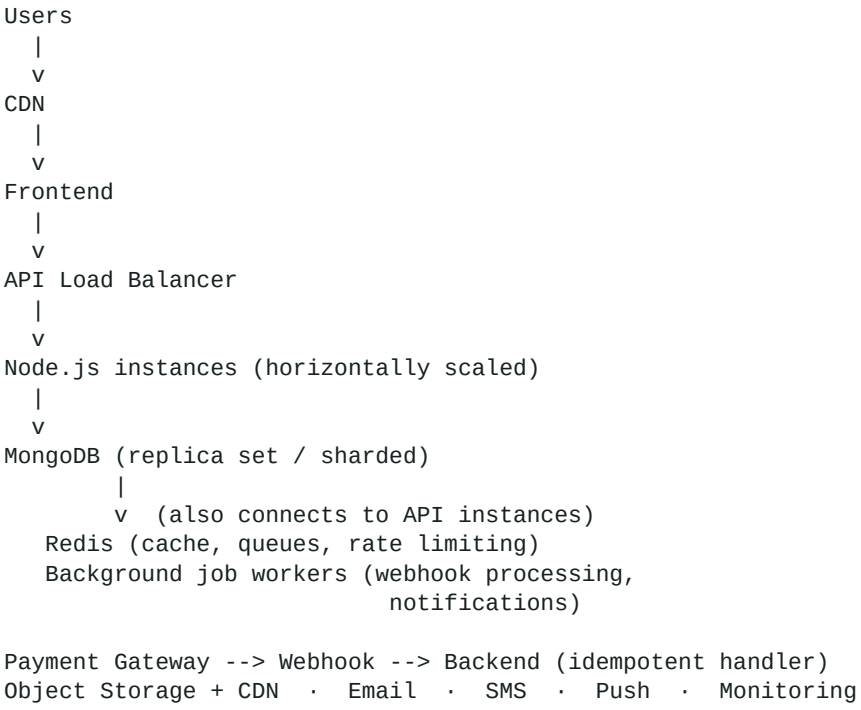


Figure 29 — Future Production-Scale Architecture

specification Section 32. This is explicitly a future-scale design, not the current MVP implementation — it introduces Redis, background job workers, and horizontal scaling that the specification's Technology Stack section (6) confirms are NOT part of the MVP.

Scalability plan (specification Section 33): at 1,000 users, a single API instance and managed MongoDB are sufficient — no caching needed. At 10,000, add index review, basic Redis caching for hot campaign pages, and a CDN for media. At 100,000, add horizontal API scaling behind a load balancer, a background job queue for notifications/webhooks, MongoDB read replicas, and rate limiting becomes essential. At

1,000,000+, database sharding, multi-region CDN, a dedicated payment-processing service, full observability, and likely splitting the monolith into payments/campaigns/notifications services.

26. End-to-End Example

The following walk-through follows one realistic scenario from the specification's persona set (Section 2) through the complete system, connecting every workflow documented above into a single story.

```
Amina creates a medical fundraising campaign
      |
      v
Amina registers & verifies her account
      |
      v
Creates campaign
      |
      v
Adds beneficiary (her son)
      |
      v
Submits campaign
      |
      v
Moderator reviews it
      |
      v
Campaign approved
      |
      v
Campaign published
      |
      v
Yusuf (diaspora donor) discovers campaign
      |
      v
Yusuf donates $50 by card
      |
      v
Payment provider processes payment
      |
      v
Webhook confirms payment (signature + idempotency checked)
      |
      v
Donation becomes confirmed
      |
      v
Campaign financial record updates
      |
      v
Amina sees the donation on her dashboard
      |
      v
Later, Amina requests withdrawal
      |
      v
Beneficiary verification is checked (hard prerequisite)
```

```
      |
      v
Admin reviews withdrawal
      |
      v
Payout is processed
```

Figure 30 — End-to-End Example — Amina's Medical Campaign

Based on the specification's persona (Amina, Mogadishu, needs funds for her son's surgery — Section 2) and real-world scenario #1 (medical/cancer treatment, Section 35).

Amina creates a medical fundraising campaign for her son's surgery. She registers on Kaalmo, verifies her account, and creates a campaign — adding a title, story, funding goal, and her son as the beneficiary. She submits the campaign for review. A moderator checks her documents, the campaign story, and the beneficiary information; finding everything consistent, the moderator approves it, and Amina publishes it and shares the link on WhatsApp.

Yusuf, a diaspora donor, discovers the campaign, reads the story, checks the verification badges, and donates $50 by card. The payment provider processes the charge; once it confirms, a webhook reaches Kaalmo's backend, which verifies the webhook's signature, checks it hasn't already been processed (idempotency), and records a confirmed PaymentTransaction. The linked Donation becomes confirmed, the campaign's raisedAmount updates, Yusuf receives a receipt, and Amina sees the new donation on her dashboard.

Later, once enough has been raised, Amina requests a withdrawal. The system checks that her son (the beneficiary) is verified and has a connected payout account — a hard prerequisite. Because beneficiary verification is confirmed, the withdrawal proceeds to admin/risk review. An admin approves it, the payout is routed through PaymentService to the chosen payout method, the provider confirms the payout, and the withdrawal is marked COMPLETED — with the campaign's financial ledger updated to reflect it.

27. Error & Alternative Flows

No workflow in Kaalmo is only a happy path. The specification defines explicit alternative outcomes at each major decision point.

```
Campaign Review
   +-- Approved
   +-- Rejected  (reason shown, may resubmit)

Payment
   +-- Success (Confirmed)
   +-- Failed  (clear retry UX)
   +-- Pending (mobile money confirmation delay)

Withdrawal
   +-- Approved
   +-- Rejected
   +-- Frozen  (pending investigation)

Fraud / Risk Score
   +-- Low       -> Allow
   +-- Medium    -> Moderator Review
```

```
+-- High      -> Freeze
+-- Critical  -> Reject / Account Flag / Admin Alert
```

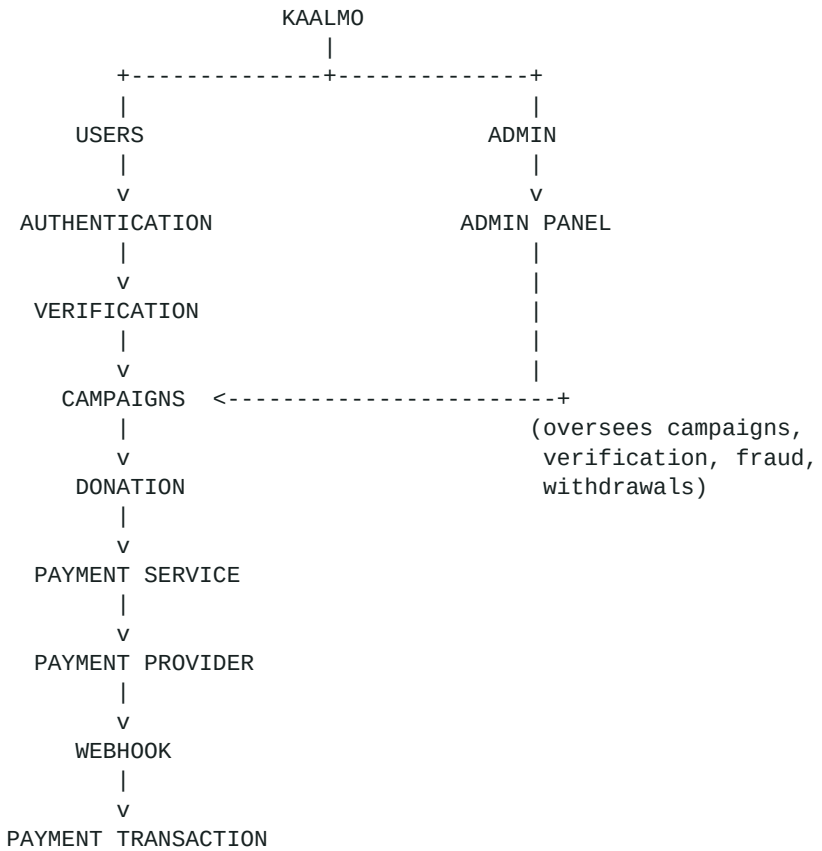
Figure 31 — Alternative Flows — Summary Across Workflows

Campaign review: a campaign that fails review is REJECTED with a reason shown to the organizer, and can be resubmitted after fixes (Section 9). **Payment:** the system must handle pending payments (mobile money confirmation delay), failed payments with a clear retry UX, and idempotent webhook handling in case of duplicate delivery (Section 14). **Withdrawal:** a request can be approved, rejected, or frozen pending investigation, with every state transition logged with actor, timestamp, and reason (Section 15). **Fraud:** a subject's risk score routes it to Allow (LOW), moderator Review (MEDIUM), Freeze (HIGH), or Reject + account flag + admin alert (CRITICAL) (Section 21).

The specification's testing strategy (Section 31) reinforces that these alternative paths are not optional to build: it calls for payment tests with "mocked provider responses for success/failure/timeout/duplicate-webhook scenarios," explicit webhook replay-protection tests, and negative authorization tests. It also flags a subtle but important reconciliation case: a payment can succeed at the provider while its webhook never arrives — "the system must reconcile via a periodic status-check job, not rely on webhooks alone" (Section 31).

28. Complete Master Diagram

This final diagram connects every subsystem documented in this report into one view — from a user's first interaction with Kaalmo through to a completed payout.



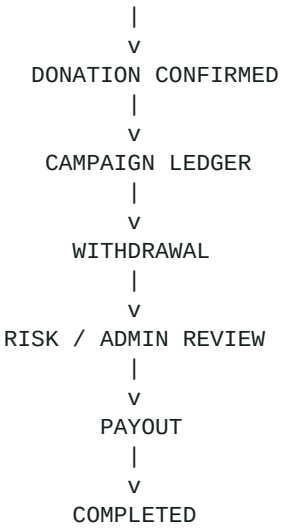


Figure 32 — Complete Master System Diagram

The full journey: Users and Admin both enter through their respective surfaces; Users flow through Authentication → Verification → Campaigns → Donation → Payment Service → Payment Provider → Webhook → Payment Transaction → Donation Confirmed → Campaign Ledger → Withdrawal → Risk/Admin Review → Payout → Completed, while Admin oversees Campaigns, Verification, Fraud/Risk, and Withdrawals directly through the Admin Panel at every stage.

29. Glossary

Term	Definition (per specification)
Donor	A user who gives money to a campaign (Section 4)
Organizer	A user who creates and runs a campaign (Section 4)
Co-organizer	A user with campaign-scoped assistance access via CampaignMembers (Sections 4, 16)
Beneficiary	The person/entity the campaign's funds are for (Section 4)
Admin	Platform operator with full administrative access (Section 4)
Moderator / Trust & Safety	Reviews campaign/user risk; no financial settings access (Section 4)
Support Agent	Handles customer support tickets; no withdrawal/fraud access (Section 4)
Campaign	A fundraiser record moving through the defined lifecycle state machine (Section 9)
Donation	A single gift from a donor to a campaign (Section 16)
Payment	High-level payment intent record (Section 16)
PaymentTransaction	Immutable ledger entry per confirmed provider event (Sections 14, 16)
PaymentWebhook	Raw webhook payload + signature verification result, retained for audit (Section 16)
Withdrawal	A request to pay out raised funds to a beneficiary/organizer (Section 15)
PayoutAccount	A mobile money or bank account registered to receive a withdrawal (Section 16)
Verification	A tiered trust check: email, phone, identity, beneficiary, payment, organization (Section 22)
FraudCase	A tracked risk record with a score and signals for a user/campaign (Section 16)
AuditLog	An immutable, append-only record of an admin or financial action (Sections 16, 20)
Notification	A system event delivered to a user via in-app, email, or SMS (Section 23)
raisedAmount	A derived, recomputable field on Campaign — never the financial source of truth (Section 17)

30. Final System Summary

```
PRODUCT: Somalia-first trusted fundraising platform
  |
  v
USERS: Donor, Organizer, Beneficiary, Co-organizer,
       Admin, Moderator, Support
  |
  v
FEATURES: Campaign lifecycle, donations, updates,
          verification, withdrawals, moderation
  |
  v
UX: Bilingual, low-bandwidth, trust-signal-forward
  |
  v
FRONTEND: React (web)
  |
  v
BACKEND: Node.js + Express, shared REST API
  |
  v
DATABASE: MongoDB, immutable financial ledger design
  |
  v
PAYMENTS: Abstracted multi-provider layer,
          mobile-money-first
  |
  v
SECURITY: JWT auth, RBAC, encrypted sensitive data,
          audit logging
  |
  v
TRUST & SAFETY: Tiered verification, risk scoring,
                reporting, moderation queue
  |
  v
ADMIN: Web-only dashboard for full platform oversight
  |
  v
TESTING: Unit, integration, payment/webhook, E2E
  |
  v
DEPLOYMENT: Containerized MVP -> horizontally
            scaled production (future)
  |
  v
GROWTH: Somalia -> diaspora hubs -> East Africa
```

Figure 33 — Final Product Blueprint

specification Section 40.

■ *Section 5 ("Web vs Mobile Strategy") and Section 24 state plainly that every role — Donor, Organizer, Beneficiary, Co-organizer, Admin, Moderator — uses a Responsive Web App, with an explicit "Admin Web: ... remains within the secure web admin dashboard" and no native mobile app anywhere in the spec's role table. Section 40's blueprint diagram and its "MOBILE: Full parity..." line, by contrast, reference a Flutter mobile app. This document follows the filename (kaalmo-web-only-spec.md) and Section 5's explicit strategy — Web Only — as authoritative, and treats the*

Flutter/mobile references in Section 40 as inherited template text rather than confirmed scope, per this document's instruction not to invent or assume functionality.

Recommended MVP scope (specification Section 40): Auth + verification (email, phone, identity, beneficiary) → campaign create/review/publish lifecycle → single working payment provider (mobile money) with donation + receipt flow → withdrawal request + admin approval → web admin dashboard covering moderation, users, payments, withdrawals → Somali/English UI → basic in-app + email notifications → responsive React web app covering donor, organizer, beneficiary, and co-organizer core flows.

Definition of Done — MVP Beta-Ready (Section 40): a real user can register, get verified, create and submit a campaign, get it approved, and publish it. A donor can donate real money via at least one payment method and receive a confirmed receipt. Campaign balance updates correctly and reconciles with the payment ledger. A beneficiary can be verified and a withdrawal can be requested, reviewed, and paid out successfully. Admin can moderate campaigns, view financials, and handle a reported campaign end-to-end. Core security controls are in place: HTTPS, hashed passwords, rate limiting, webhook signature verification, input validation on all endpoints. Somali and English UI both work across all core screens. Automated tests cover auth, campaign lifecycle, donation flow, and payment webhook handling.

Kaalmo is not a GoFundMe clone — it borrows the proven mechanics of crowdfunding (campaign → story → donate → update) but rebuilds the trust and payment layer around Somali realities: mobile-money-first payments, bilingual Somali/English UX, and a verification system suited to Somali community trust culture (specification Section 39).

— End of Document —